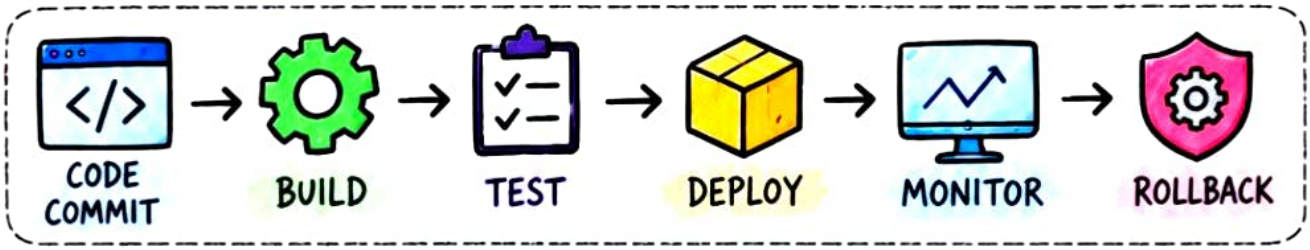
# DEV → QA → PROD

# ZERO TO EVERYTHING

## CI/CD & PRODUCTION ARCHITECTURE




## WHAT YOU WILL LEARN

- DEV Environment
- QA Environment
- PROD Environment
- CI/CD Pipeline
- Docker & Containerization
- Kubernetes / EKS
- Artifact & Image Management
- Approval & Release Process
- Monitoring & Logging
- Rollback Strategy
- Security Best Practices
- Cost Optimization
- High Availability & Scalability
- Troubleshooting & Alerts
- Real-World Production Flow

## WHY THIS GUIDE?

Understand the complete journey of an application from code commit to live production with industry best practices.

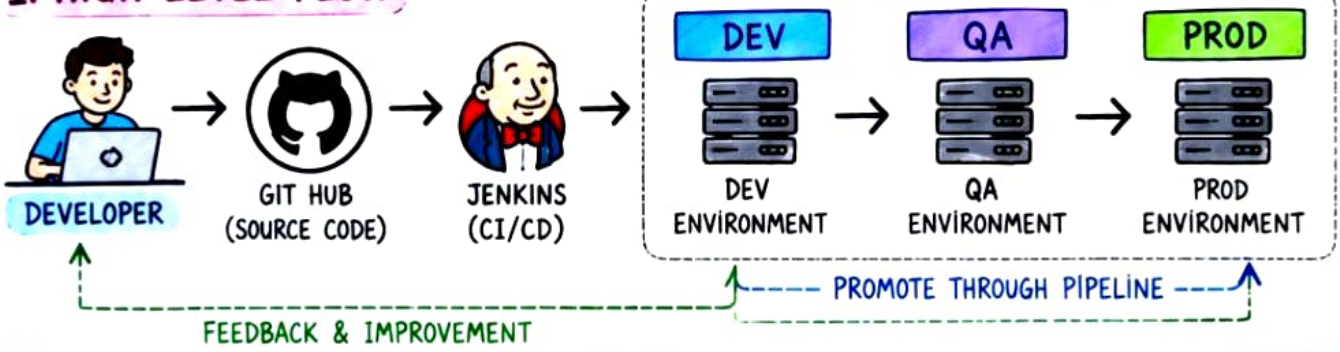
By - Rakesh Kumar ♥

# DEV → QA → PROD

## ZERO TO EVERYTHING

Complete DevOps Environment Flow & Architecture Overview

### 1. HIGH LEVEL FLOW




### 2. ENVIRONMENTS EXPLAINED

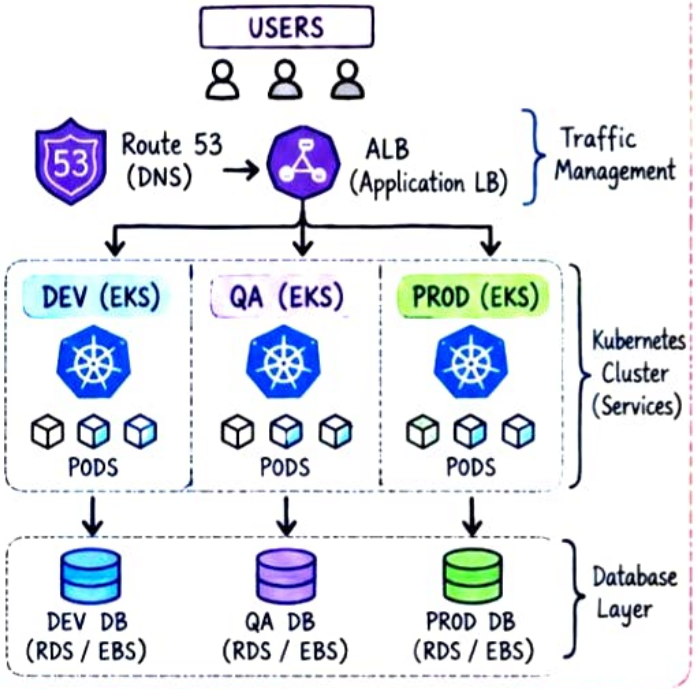
**DEV**
- Development & Unit Testing
- Feature build & self testing
- Deployed by Developers

**QA**
- Integration Testing
- Functional, API, UAT
- Bug fixing & validation

**PROD**
- Live Application
- Real Users
- High Availability & Monitoring

Code moves from DEV → QA → PROD only after successful validation.

### 3. ARCHITECTURE OVERVIEW




### 4. KEY COMPONENTS

| SOURCE CODE (GitHub) | CI/CD (Jenkins) | ARTIFACT / IMAGE (Nexus / ECR) | DEPLOYMENT (RDS / ECS) | MONITORING (CloudWatch / Prometheus) | SECURITY (IAM, SSM, WAF) |
|---|---|---|---|---|---|

### 5. PIPELINE SUMMARY




### 6. OBJECTIVE

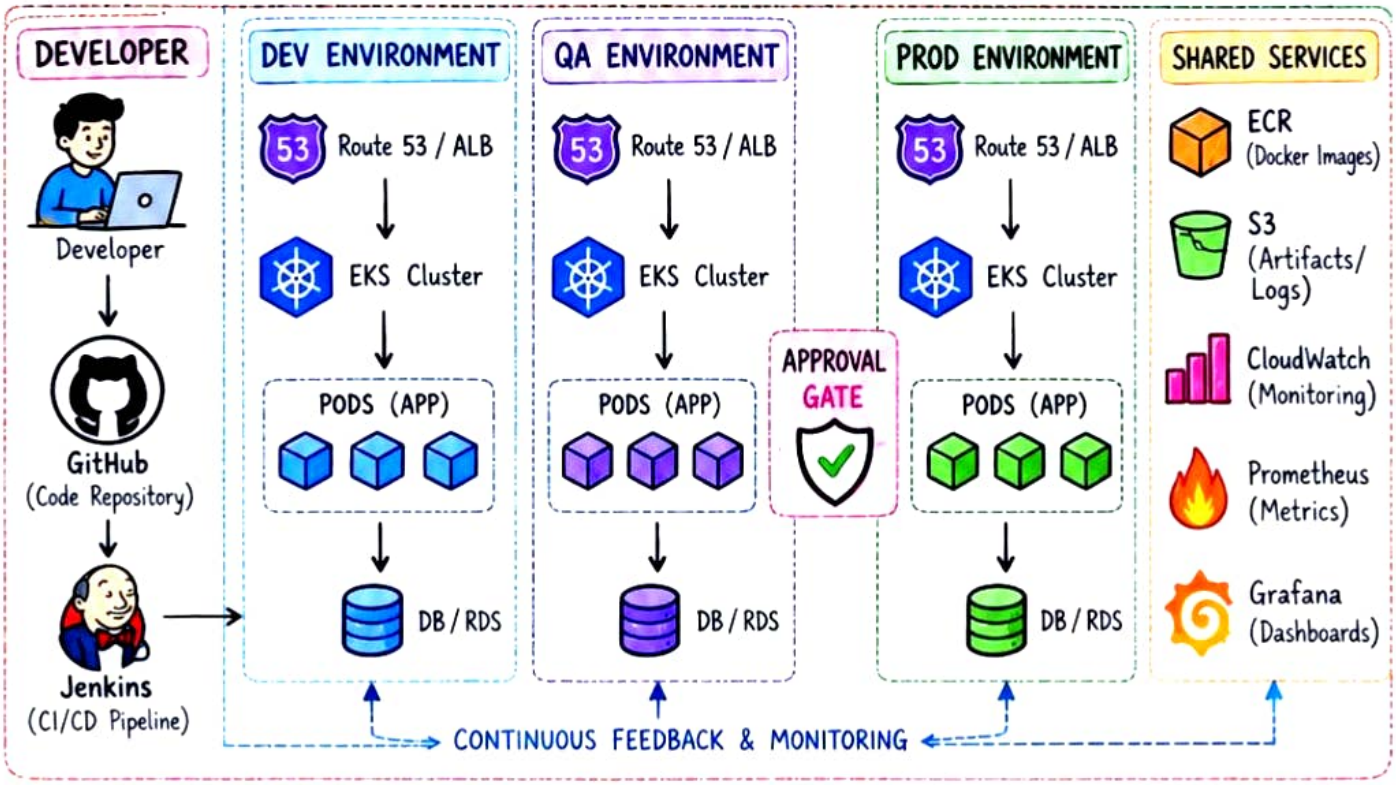


- [x] Deliver feature faster
- [x] Maintain quality
- [x] Ensure security
- [x] Achieve high availability
- [x] Reduce downtime & cost

# DEV → QA → PROD

## COMPLETE ARCHITECTURE OVERVIEW

High Level End-to-End Architecture from Developer to Production



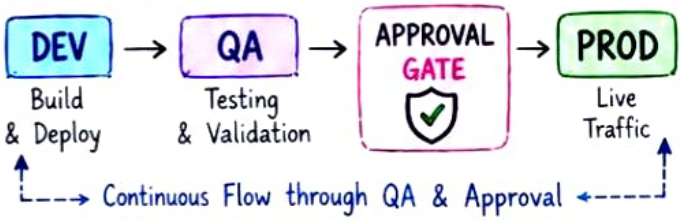

### 1. ARCHITECTURE FLOW

- Developer writes code and pushes to GitHub.
- Jenkins picks the code and runs CI/CD pipeline.
- Code is built, tested and Docker image is created.
- Image is stored in ECR, then deployed to DEV.
- After validation, promoted to QA for testing.
- After QA approval, promoted to PROD.
- Monitoring, logging and alerts are active 24/7.

### 2. KEY TRAFFIC FLOW




### 3. PROMOTION FLOW




### 4. COMPONENTS USED

| | | |
|---|---|---|
| GitHub | – | Source code management |
| Jenkins | – | CI/CD automation |
| ECR | – | Store Docker images |
| EKS | – | Kubernetes managed by AWS |
| ALB | – | Application Load Balancer |
| RDS | – | Managed Database |
| S3 | – | Storage for artifacts, logs, backup |
| CloudWatch | – | Monitoring, Logs, Alarms |
| Prometheus | – | Metrics collection |
| Grafana | – | Dashboards & Visualization |

### 5. BENEFITS

- Separation of environments (Safe & Isolated)
- Quick and automated delivery (CI/CD)
- Reliable, scalable and secure infrastructure
- High availability and cost efficient
- Real-time monitoring, logging and alerting
- Faster release with rollback capability

# DEV ENVIRONMENT

The foundation where developers build, test & validate before promoting to QA.

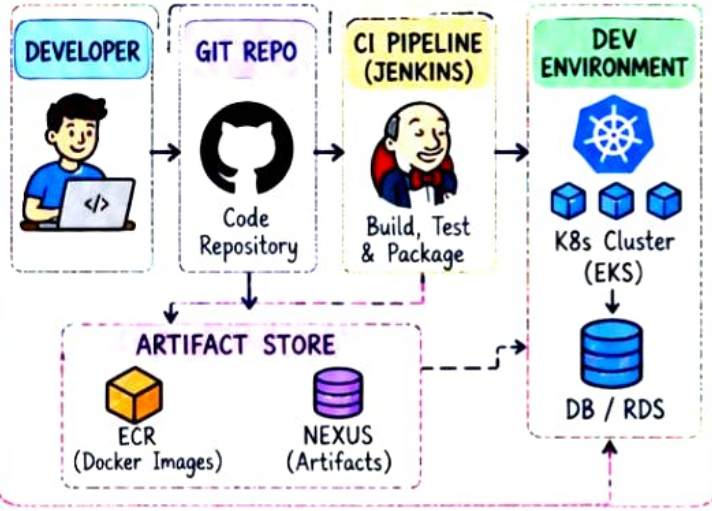
## 1. WHAT IS DEV ENVIRONMENT?

DEV

- Dev (Development) environment is the first stage of the SDLC.
- Used by developers to build new features, fix bugs & do unit testing.
- Changes are frequent & isolated from other environments.
- Goal: Fast feedback, innovation & early validation.

Everything starts here before moving towards QA and Production.

## 2. DEV ENVIRONMENT ARCHITECTURE




## 3. DEV ENVIRONMENT COMPONENTS

| | |
|---|---|
| EKS Cluster | Run containerized applications |
| ECR | Store Docker images |
| RDS / DB | Application database |
| CloudWatch | Monitoring & Logs |
| IAM | Access & permissions |
| S3 Bucket | Store configs, artifacts, logs |
| ALB / NLB | Traffic distribution |

## 4. DEV ENVIRONMENT DETAILS

| | |
|---|---|
| Purpose | Development & Unit Testing |
| Users | Developers & Testers |
| Access | Restricted (IAM + Roles) |
| Life Cycle | Short lived / Resettable |
| Data | Dummy / Masked data |
| Updates | Multiple times a day |
| Monitoring | Basic monitoring enabled |
| Cost Focus | Low cost, right-sized resources |

## 5. TYPICAL DEV ENVIRONMENT SETUP




Fast feedback loop helps developers catch issues early and push quality code to QA.

## 6. BEST PRACTICES FOR DEV ENVIRONMENT

- Keep environments isolated.
- Use Infrastructure as Code (Terraform / CloudFormation).
- Automate build, test & deployments.
- Use dummy data for new changes.
- Use right-sized resources to control cost.
- Enable monitoring & alerts.
- Clean up unused resources regularly.

## 7. DEV vs QA vs PROD (QUICK VIEW)

| Aspect | DEV | QA | PROD |
|---|---|---|---|
| Purpose | Development | Testing & Validation | Live Users |
| Data | Dummy / Masked | Test Data | Real Data |
| Access | Developers | QA Testers | Restricted |
| Changes | Very Frequent | Controlled | Minimal |
| Goal | Build & Validate Fast | Ensure Quality & Stability | High Availability & Performance |

AWS DEVOPS NOTES SERIES

# GIT & BRANCHING STRATEGY

A well-structured Git workflow ensures smooth collaboration, version control and reliable CI/CD pipeline.

## 1. WHY GIT?

- Distributed Version Control System
- Track entire history of changes
- Collaboration with multiple developers
- Supports branching & merging
- Rollback to any previous version
- Integrates with CI/CD pipelines

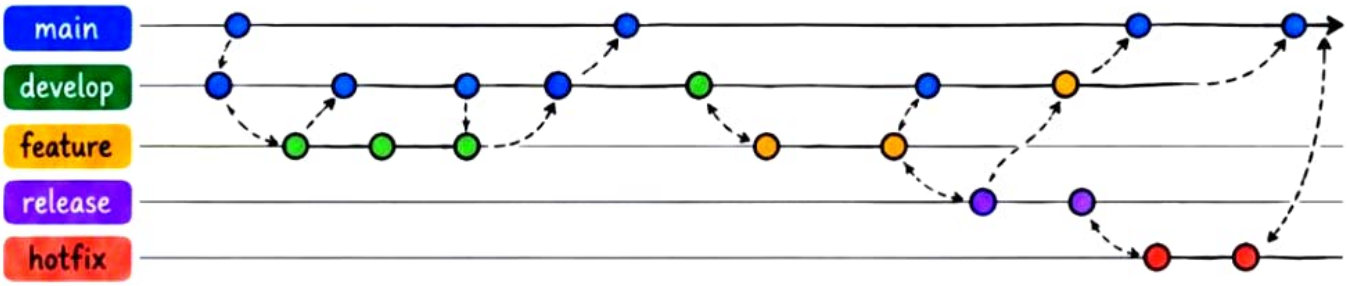
## 2. BRANCHING MODEL (OVERVIEW)

| MAIN | DEVELOP | FEATURE | HOTFIX |
|---|---|---|---|
| Production Ready Code | Integration of features by Developers | New feature development (Short-lived) | Critical fixes for Production |

Trunk Based Development with Feature Branch Workflow.

## 3. BRANCHING WORKFLOW

Commit · Merge · Release · Hotfix · ----> Branch flow




**Feature Branch** — Create from develop branch for new features or tasks.

**Merge to Develop** — Once feature is complete, create PR & merge to develop.

**Release Branch** — Create release branch before the UAT. Merge to main after testing.

**Merge to Main** — After approval, merge release branch to main.

**Hotfix Branch** — Create hotfix from main for critical production issues.

## 4. COMMON GIT COMMANDS

| | |
|---|---|
| `git clone <repo-url>` | Clone repository |
| `git branch` | Create a new branch |
| `git checkout <branch>` | Switch branch |
| `git switch <branch>` | Switch branch (new) |
| `git pull origin <branch>` | Pull latest changes |
| `git push origin <branch>` | Push branch to remote |
| `git merge <branch>` | Merge branch into current |
| `git status` | Check working status |
| `git log --oneline` | View commit history |
| `git stash` | Save temporary changes |

## 5. BEST PRACTICES

- Commit small & meaningful changes.
- Write clear commit messages.
- Create Pull Request (PR) for code review.
- Keep develop branch stable.
- Delete merged feature branches.
- Protect main branch (PR, review, policies).
- Automate build, test & deployments.
- Always sync your branch before pushing.

## 6. BRANCH NAMING CONVENTIONS

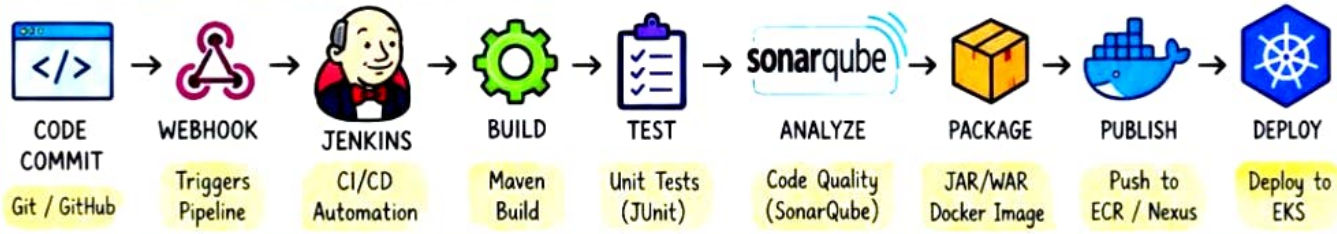
| feature/<name> | bugfix/<name> | release/<version> | hotfix/<name> |
|---|---|---|---|
| feature/login | bugfix/payment-error | release/v1.2.0 | hotfix/critical-issue |
| For new feature development | For bug fixes | For release preparation | For production fixes |

Consistent naming makes collaboration easy & organized.

# JENKINS CI PIPELINE

Automate Build, Test, Code Quality & Package Delivery with Jenkins

## 1. JENKINS CI PIPELINE OVERVIEW



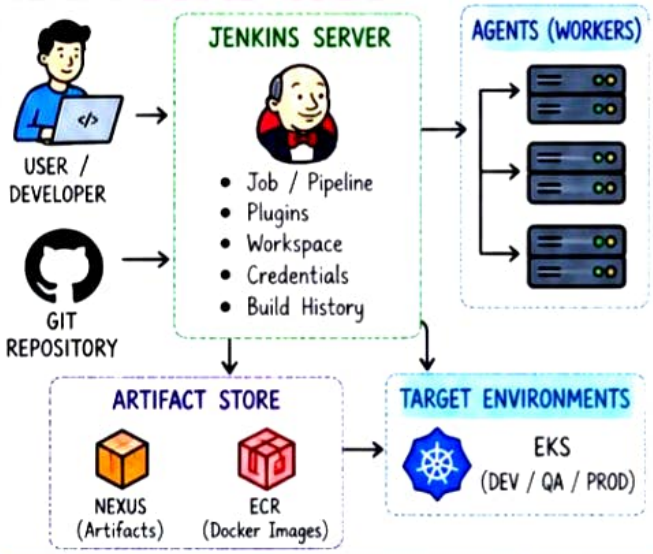

One code commit triggers the pipeline and delivers a tested & packaged artifact ready for deployment.

## 2. PIPELINE STAGES (DETAILS)

| | | | |
|---|---|---|---|
| 1 | Code Checkout | → | Pull code from Git repository |
| 2 | Build | → | Compile source code & create build artifact (Maven) |
| 3 | Unit Test | → | Run unit tests (JUnit / TestNG) |
| 4 | Code Quality | → | Static analysis using SonarQube |
| 5 | Package | → | Create artifact (JAR/WAR) |
| 6 | Docker Build | → | Build Docker image |
| 7 | Push to Registry | → | Push image to ECR / Nexus |
| 8 | Deploy to Dev | → | Deploy to DEV (EKS) for validation |

If any stage fails, pipeline stops and notifies the team.

## 3. JENKINS ARCHITECTURE




## 4. JENKINS PIPELINE (SAMPLE FLOW)

```
pipeline {
  agent any
   stages {
    stage('Checkout')  { steps { git 'https://github.com/org/repo.git' } }
    stage('Build')     { steps { sh 'mvn clean package -DskipTests' } }
    stage('Unit Test') { steps { sh 'mvn test' } }
    stage('SonarQube') { steps { withSonarQubeEnv('sonar') {
                                 sh 'mvn sonar:sonar' } } }
    stage('Package')   { steps { archiveArtifacts artifacts: 'target/*.jar' } }
    stage('Docker Build'){ steps { sh 'docker build -t myapp:$BUILD_NUMBER .' } }
    stage('Push')      { steps { sh 'docker tag myapp:$BUILD_NUMBER
                                 sh 123456789.dkr.ecr.amazonaws.com/myapp:$BUILD_NUMBER'
                                 sh 'docker push 123...amazonaws.com/myapp:$BUILD_NUMBER'} }
    stage('Deploy to Dev'){ steps { sh 'kubectl apply -f k8s/dev/' } }
  }
  post {
    success { echo 'Build Successful! ✅' }
    failure { echo 'Build Failed! ❌' }
  }
}
```

## 5. TOOLS & INTEGRATIONS

| | | |
|---|---|---|
| Jenkins | – | CI/CD Automation |
| Git / GitHub | – | Source Code Management |
| sonarqube | – | Code Quality Analysis |
| Maven | – | Build Automation |
| Docker | – | Containerization |
| ECR / Nexus | – | Artifact / Image Repository |
| Kubernetes (EKS) | – | Deployment |
| Slack / Email | – | Notifications & Alerts |

## 6. ADVANTAGES

- Automated and consistent delivery
- Early detection of issues
- Improves code quality with tests & analysis
- Reduces manual work & human errors
- Faster feedback & quick releases

## 7. BEST PRACTICES

- Keep pipeline simple & modular
- Use credentials & secrets securely
- Fail fast & notify early
- Clean workspace after build
- Version everything (code, infra, pipeline)
- Use parallel stages where possible

## 8. TIPS

- Use parallel stages for faster builds
- Cache dependencies to save time
- Use Jenkinsfile (Pipeline as Code)
- Monitor build health regularly
- Keep plugins updated
- Setup notifications (Slack / Email)

# DOCKER + ECR / NEXUS

Containerize Applications, Store Securely & Manage Artifacts Efficiently

## 1. DOCKER OVERVIEW

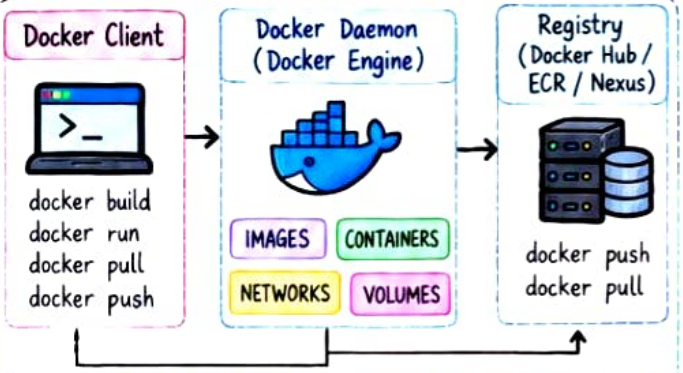


- Docker packages an application with its dependencies into a container.
- Containers run the same way in any environment.
- Lightweight, portable and scalable.
- Speeds up development and deployment.

### KEY CONCEPTS

- **Image** – Read-only template to create containers
- **Container** – Running instance of an image
- **Dockerfile** – Instructions to build an image
- **Registry** – Stores and distributes images
- **Volume** – Persistent storage for containers
- **Docker Compose** – Run multi-container applications

## 2. DOCKER ARCHITECTURE




Docker Client communicates with Docker Daemon, which builds, runs and manages containers & images.

## 3. DOCKER WORKFLOW




### COMMON DOCKER COMMANDS

```
docker build -t myapp:1.0 .                  # Build image
docker images                                # List images
docker run -d -p 8080:8080 myapp:1.0         # Run container
docker ps                                    # List running containers
docker stop <container_id>                   # Stop container
docker rm <container_id>                     # Remove container
docker rmi <image_id>                        # Remove image
docker push <repo>:<tag>                     # Push image to registry
docker pull <repo>:<tag>                     # Pull image from registry
```

## 4. DOCKERFILE EXAMPLE (JAVA APP)

```
# Use official OpenJDK image
FROM eclipse-temurin:17-jdk-alpine          # Base image
WORKDIR /app                                # Working directory
COPY target/myapp.jar app.jar               # Copy jar file
EXPOSE 8080                                 # Expose port
ENTRYPOINT ["java", "-jar", "app.jar"]      # Run the app
```

### Build & Run

```
docker build -t myapp:1.0 .                 # Build image
docker run -d -p 8080:8080 myapp:1.0        # Run container
docker ps                                   # Check running container
```

## 5. AMAZON ECR (ELASTIC CONTAINER REGISTRY)



Amazon ECR

- Fully managed Docker container registry.
- Store, manage and deploy container images securely on AWS.
- Integrates with IAM, CloudTrail & Lambda.
- Highly available, scalable and reliable.

### Basic ECR Commands

```
aws ecr get-login-password --region <region> | \
  docker login --username AWS --password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com
docker tag myapp:1.0 <account-id>.dkr.ecr.<region>.amazonaws.com/myapp:1.0
docker push <account-id>.dkr.ecr.<region>.amazonaws.com/myapp:1.0
aws ecr describe-repositories --region <region>
```

## 6. NEXUS (ARTIFACT REPOSITORY MANAGER)



Nexus

- Universal repository manager.
- Supports Docker, Maven, npm, Python etc.
- Store and manage artifacts.
- Host private Docker registry.
- Control access, versions and retention policies.

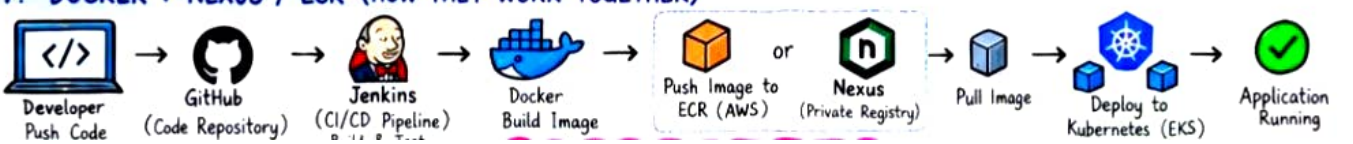
### Supported Repositories

- Docker (Docker Images)
- Maven (JAR/WAR)
- npm (Node.js packages)
- Python (PyPI packages)
- Helm (Charts)
- Generic (any files)

### Key Features

- Self-hosted private registry
- Access control & security
- Versioning & artifact management
- Repository cleanup policies
- High availability
- Integrates with CI/CD tools

## 7. DOCKER + NEXUS / ECR (HOW THEY WORK TOGETHER)



# QA ENVIRONMENT & TESTING

Validate, Verify & Ensure Quality Before Promoting to Production

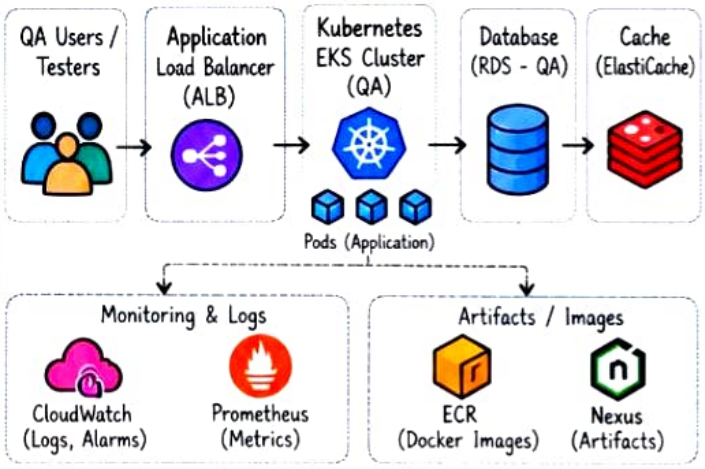
## 1. QA ENVIRONMENT OVERVIEW

**QA**
(Quality Assurance)

- QA environment is used to validate the build in a production-like environment.
- Helps to identify bugs, functional issues, integration issues and performance issues early.
- Only after successful QA, the build is approved for Production deployment.
- Goal: Ensure application quality, stability and user satisfaction.

Think of QA as a mirror of Production to ensure we deliver bug-free & reliable software.

## 2. QA ENVIRONMENT ARCHITECTURE (EXAMPLE)




QA infrastructure is isolated and safe to test new builds without affecting Production.

## 3. KEY COMPONENTS IN QA

| | |
|---|---|
| EKS Cluster | Runs application containers (same as PROD) |
| Load Balancer | Distributes traffic to application pods |
| RDS (QA) | QA database with test data set |
| ElastiCache | Caching layer for performance testing |
| S3 (Storage) | Stores logs, reports, test data, artifacts |
| CloudWatch | Monitoring, logs, metrics and alerts |
| ECR / Nexus | Images from ECR or artifacts from Nexus |
| Secrets Manager | Store secrets, credentials, config |
| VPC (Isolated) | Isolated network, subnets, security groups |

## 4. TESTING TYPES PERFORMED IN QA

**Functional Testing**
- Validate features and workflows.

**UI / UX Testing**
- Validate frontend, layouts and user experience.

**Integration Testing**
- Validate integration with other systems and services.

**Performance Testing**
- Load, stress and spike testing using JMeter, k6 or Locust.

**Regression Testing**
- Ensure new changes don't break existing functionality.

**Security Testing**
- SAST, DAST, dependency scan and vulnerability scan.

**API Testing**
- Test REST APIs using Postman, Newman or RestAssured.

**Compatibility Testing**
- Cross-browser, device and OS compatibility testing.

## 5. QA FLOW




Only after all critical and major issues are resolved, QA approves the build for Production promotion.

## 6. TEST DATA MANAGEMENT

- Use masked / anonymized data.
- Refresh data regularly from production snapshots (backup) in a safe way.
- Provide least-privilege access to test data.
- Maintain separate test users and roles.
- Clean up test data after execution.

## 7. TOOLS & INTEGRATIONS USED IN QA

| Postman | Selenium | JMeter / k6 | SonarQube | Nexus | CloudWatch |
|---|---|---|---|---|---|
| (API Testing) | (UI Automation) | (Performance Testing) | (Code Quality Analysis) | (Artifact Repository) | (Monitoring & Logs) |

## 8. QA ENVIRONMENT BEST PRACTICES

- Keep QA environment close to Production.
- Automate test execution wherever possible.
- Run tests in parallel to save time.
- Track code coverage and quality metrics.
- Manage and track defects using a tool (e.g., JIRA).
- Monitor environment health (logs, metrics, alerts).
- Collaborate closely between Dev, QA and Ops.

# QA → PROD APPROVAL FLOW

Controlled, Secure & Auditable process to promote stable builds from QA to Production.

## 1. QA → PROD APPROVAL FLOW OVERVIEW

1. **Build Ready in QA** — QA build is tested and stable
2. **QA Testing Completed** — Functional, Integration, Performance & Security testing done
3. **Approval Request Raised** — Create request in tool (Jira / ServiceNow / Custom Portal)
4. **Review & Approvals** — Multiple stakeholders review and approve
5. **Approved for PROD** — All mandatory approvals completed
6. **Deploy to Production** — Build is deployed in Production
7. **Post-Deploy Verification** — Verify application health & monitoring

No direct production access. All changes go through approval gates.

## 2. WHO CAN APPROVE ?

- **Developer (Initiator)** — Raise request for promotion
- **QA Lead / Tester** — Confirms testing is complete & quality criteria met
- **DevOps / Release Manager** — Verifies build, config, infra, rollback plan
- **Product Owner / Business** — Confirms business readiness
- **Security / Compliance (Optional)** — Security checks for production
- **Change Advisory Board (CAB) (Optional)** — Final go-ahead for production

## 3. APPROVAL GATES (DETAILS)

| # | GATE | PURPOSE | BY WHOM | WHAT IS CHECKED? |
|---|---|---|---|---|
| 1 | QA Completion | Ensure build is thoroughly tested | QA Lead / Testers | Test cases results, Bugs, UAT results, Test data |
| 2 | Technical Review | Verify technical readiness | DevOps / Release Manager | Build version, Config changes, Infra impact, Rollback plan |
| 3 | Business Approval | Ensure business readiness | Product Owner / Business | Business impact, Go-Live readiness, Communication |
| 4 | Security & Compliance | Ensure security & compliance | Security / Compliance (Optional) | Vulnerability scan, Secrets, Policies, Compliance checks |
| 5 | CAB Approval (Optional) | Final approval for change | CAB Members | Risk, Impact, Backout plan, Maintenance window |

All approvals are recorded & auditable (Jira / ServiceNow / Change Management tool).

## 4. APPROVAL REQUEST PROCESS

| | |
|---|---|
| Create Approval Request | Raise request in tool (Jira / ServiceNow / Custom Portal) |
| Provide Details | Change Description, Build Version / Image, Environment (PROD), Impact & Rollback Plan |
| Notify Approvers | Automatic notifications sent to all approvers |
| Wait for Approvals | Each approver reviews & takes action |
| Final Decision | If all mandatory approvals are complete → Approved<br>Else → Rejected / Changes |

## 5. APPROVAL STATES

- **PENDING** — Request raised, waiting for approvers
- **IN REVIEW** — Approver is reviewing the request
- **APPROVED** — All required approvals completed
- **REJECTED** — Request rejected, needs changes
- **CANCELLED** — Request cancelled by initiator

**Policy Example:**
- All approvals must be from authorized people only.
- Minimum 2-3 approvals required for PROD deployment.
- Rejection requires reason & resolution.

## 6. TOOLS USED (EXAMPLES)

- **Jira** (Approval & Change Mgmt)
- **ServiceNow** (Change Management)
- **GitHub** (Source Code)
- **AWS CodePipeline** (Build & Deploy)
- **Slack / Email** (Notifications)

## 7. BEST PRACTICES

- Define clear approval policies.
- Keep approval steps minimum but effective.
- Use least privilege access.
- Ensure all approvals are time-bound.
- Maintain audit trail of all approvals.
- Include rollback plan in every request.
- Automate notifications & reminders.

## 8. TIPS

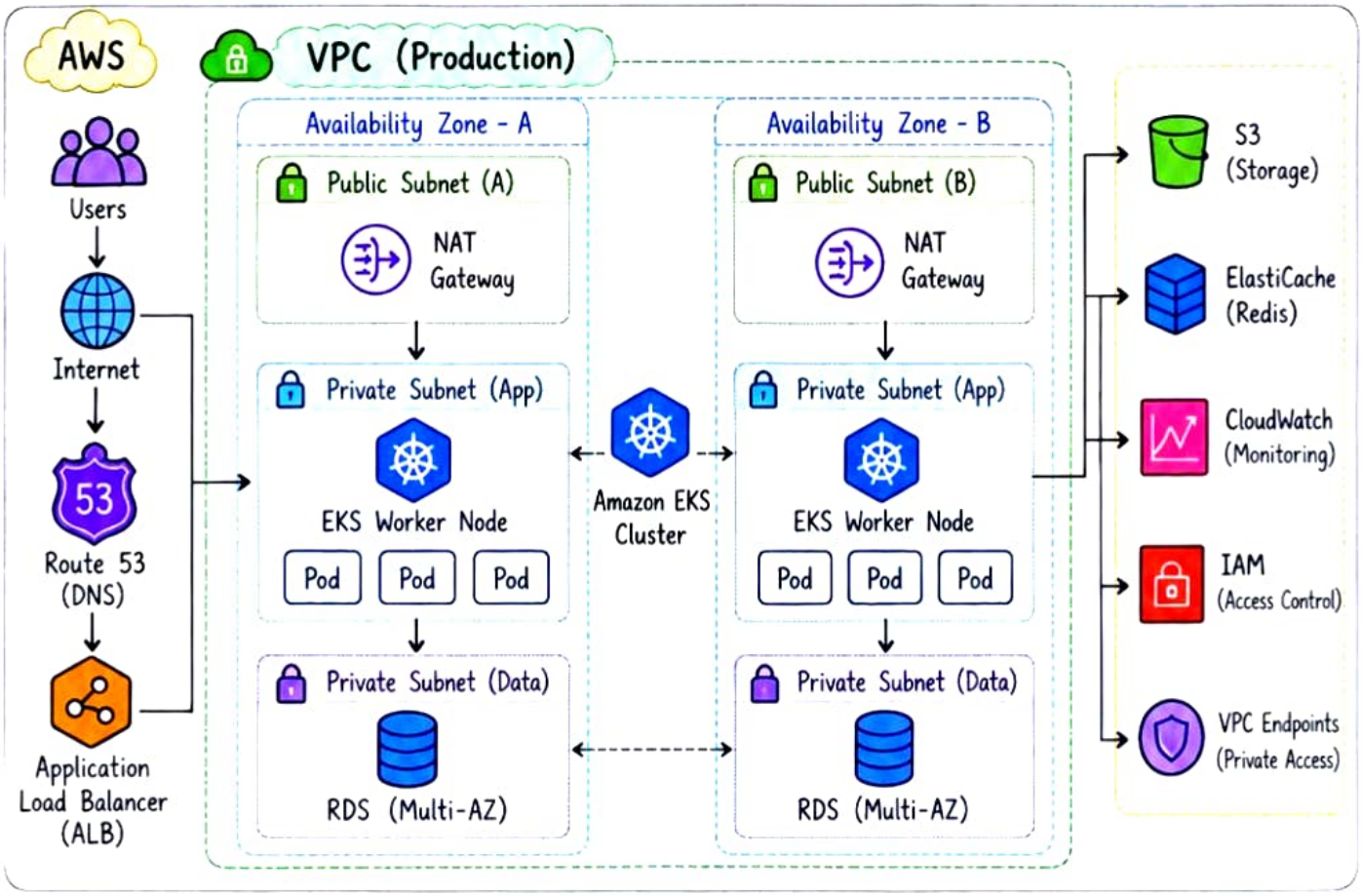
- Automate notifications to avoid delays.
- Make approval dashboard in tool.
- Don't allow direct PROD deployment.
- Regularly review & optimize the flow.
- Monitor deployment after approval (application health, logs, metrics).
- Keep approvers aware & accountable.

CLOUD & DEVOPS NOTES

EKS / AWS

# PROD ARCHITECTURE – EKS/AWS

Highly Available, Scalable & Secure Production Architecture on AWS using EKS




## 1. CORE COMPONENTS

- Amazon EKS – Managed Kubernetes service
- ALB – Distributes traffic to services
- Route 53 – DNS management
- VPC – Isolated network for production
- Subnets – Public (NAT) and Private (App & DB)
- EKS Worker Nodes – Runs application pods
- RDS (Multi-AZ) – Highly available database
- S3 – Stores logs, backups, artifacts, etc.
- ElastiCache (Redis) – Caching for performance
- CloudWatch – Monitoring, metrics & logs
- IAM – Access control & permissions
- VPC Endpoints – Secure access to AWS services

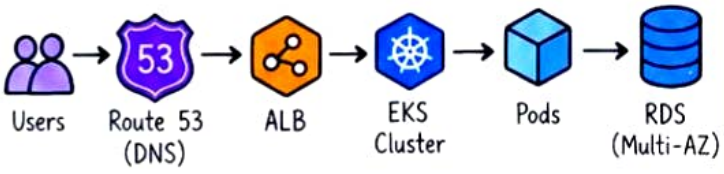
## 2. NETWORK LAYOUT

- VPC with 2 Availability Zones (AZ-A & AZ-B)
- Public Subnets – ALB, NAT Gateway
- Private Subnets (App) – EKS Worker Nodes (Pods run here)
- Private Subnets (Data) – RDS in Multi-AZ setup
- Traffic from Internet → Route 53 → ALB → EKS Services in private subnets
- NAT Gateway provides outbound internet access for private subnets.

## 3. SECURITY HIGHLIGHTS

- VPC with public & private subnets
- Security Groups to control traffic
- IAM Roles for Service Accounts (IRSA) for secure access to AWS services
- Secrets stored in AWS Secrets Manager / Parameter Store
- VPC Endpoints for S3, ECR, etc. (private access – no internet)
- KMS encryption for data
- Regular monitoring with CloudWatch
- Highly available & fault tolerant setup

## 4. TRAFFIC FLOW




## 5. KEY BENEFITS

- High Availability – Multi-AZ setup
- Scalability – Auto scaling with EKS & ALB
- Security – Isolated network, IAM, SG, VPC Endpoints
- Reliability – Managed services, monitoring, backups
- Cost Optimization – Uses AWS managed & scalable services
- Easy Deployment – Rolling updates with zero downtime

AWS DEVOPS NOTES SERIES

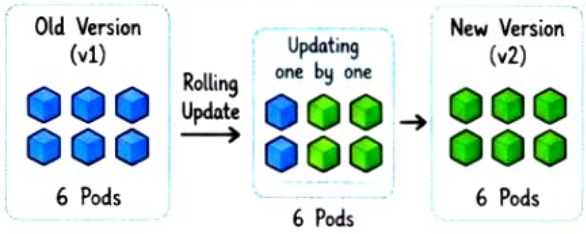
# DEPLOYMENT STRATEGIES

Different ways to release changes to Production with zero / low downtime & minimum risk.

## 1. ROLLING UPDATE (Default in Kubernetes)

- New version is rolled out gradually by replacing old pods one by one.
- Ensures zero downtime.
- Default strategy in Kubernetes (Deployments).

How it works



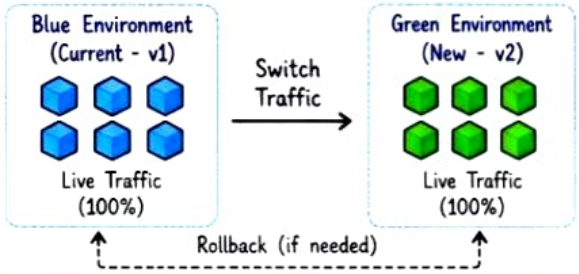

**Pros**
- Zero downtime
- Easy to implement
- Built-in in Kubernetes
- Stable and reliable

**Cons**
- Issues may take time to detect (partial rollout)
- Not suitable for breaking changes

## 2. BLUE / GREEN DEPLOYMENT

- Two identical environments (Blue & Green).
- Traffic switched from Blue to Green after validation.
- Instant rollback by switching back.

How it works



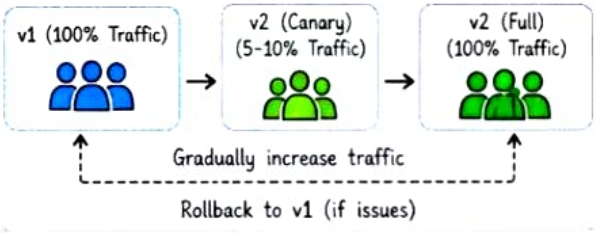

**Pros**
- Zero downtime
- Easy rollback
- Full validation before switch
- Good for high critical apps

**Cons**
- Double infrastructure cost
- Requires extra resources
- Switching requires coordination

## 3. CANARY DEPLOYMENT

- New version (v2) released to a small subset of users.
- If stable, gradually increase traffic to new version.
- Reduces risk of impact.

How it works




**Pros**
- Low risk
- Real user validation
- Granular traffic control
- Good for large applications

**Cons**
- Complex to set up
- Needs traffic routing (ALB / Istio)
- Monitoring is important

## 4. RECREATE DEPLOYMENT

- Old version is completely stopped before new version starts.
- Simple but causes downtime.
- Use only for non-critical applications.

How it works



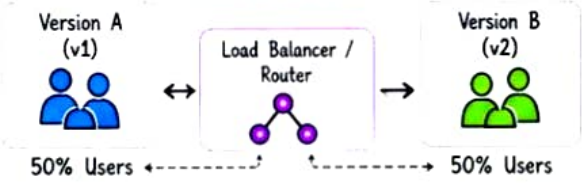

**Pros**
- Very simple
- No overlapping resources

**Cons**
- Downtime during deployment
- Not suitable for production/ critical apps

## 5. A/B DEPLOYMENT

- Two versions (A & B) live at the same time.
- Users split between versions.
- Helps in testing, experiments & marketing (feature toggles).

How it works




**Pros**
- Compare performance
- Easy rollback
- Great for experiments (Feature Testing, Marketing)

**Cons**
- High infrastructure cost
- Complex to manage

## When to Use Which Strategy?

- Rolling Update → Default, most cases (recommended)
- Blue / Green → High critical apps, need instant rollback
- Canary → Large apps, new features, risk control
- Recreate → Non-critical apps (simple)
- A/B → Experiments, feature testing, marketing

## Tools / Technologies




## Best Practices

- Choose strategy based on app criticality
- Automate deployment (CI/CD)
- Use health checks & monitoring
- Keep rollback plan ready
- Test in non-prod before production
- Monitor metrics, logs and user impact
- Use feature flags for safe releases

AWS DEVOPS NOTES SERIES

# MONITORING & LOGGING

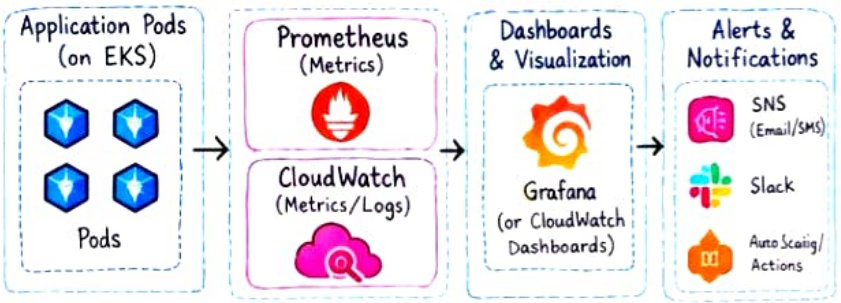
Monitor • Collect Logs • Analyze • Alert • Keep Your Systems Healthy

## 1. MONITORING OVERVIEW

- Monitoring helps track the health, performance and availability of your applications & infrastructure.
- It provides real-time insights, detects issues early and ensures high availability & reliability.
- Logs provide detailed information about what happened and why.
- Together, monitoring + logging give full visibility and faster troubleshooting.

**Goal:** Keep your systems healthy, performant and secure.

## 2. AWS / EKS MONITORING ARCHITECTURE



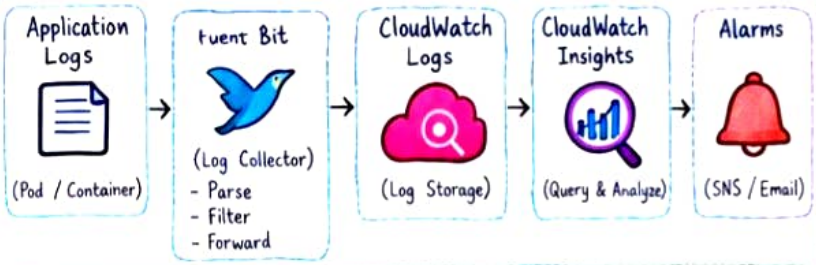

Application Pods → Prometheus/CloudWatch → Dashboards → Alerts → Notifications

## 3. KEY METRICS TO MONITOR

| | | |
|---|---|---|
| CPU Utilization | – | Pod / Node / Cluster |
| Memory Usage | – | Pod / Node / Cluster |
| Disk Usage | – | Node / EBS / Persistent Volumes |
| Network | – | In/Out Traffic, Load Balancer |
| Pod Count | – | Running / Pending / Failed |
| 5xx Error Rate | – | Application errors |
| Response Time | – | Latency / Duration |

**Goal:** Metrics help you detect issues, optimize performance and plan capacity.

## 4. LOGGING ARCHITECTURE




Collect → Process → Store → Analyze → Alert

## 5. TOOLS & SERVICES

| | | | | |
|---|---|---|---|---|
| CloudWatch (Metrics & Logs) | Prometheus (Metrics) | Grafana (Dashboards) | Fluent Bit (Log Collector) | Kinesis (Streaming Logs) |
| ELK Stack (Logs Analysis) | X-Ray (Tracing) | S3 (Log Storage) | SNS (Notifications) | IAM (Access Control) |

## 6. EXAMPLE CLOUDWATCH ALARMS

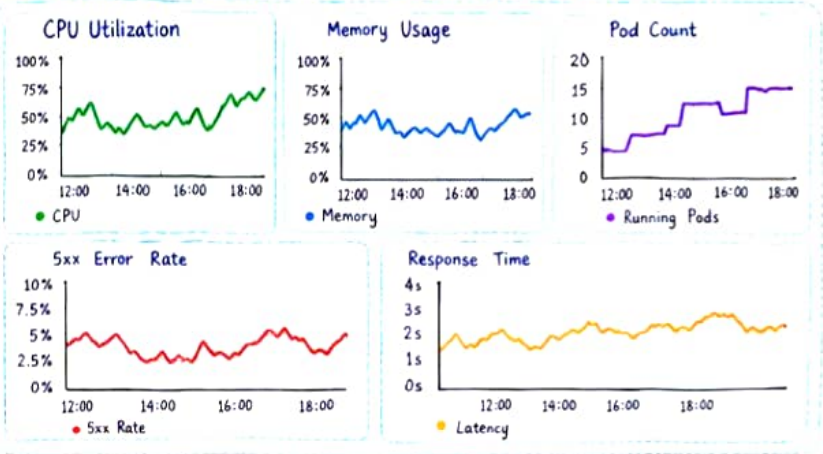
| Metric | Threshold | Action |
|---|---|---|
| CPU Utilization | > 80% | SNS Notification |
| Memory Utilization | > 85% | Auto Scaling / SNS |
| 5xx Error Rate | > 5% | SNS Notification |
| Pod Count (Low) | < 2 | SNS Notification |
| Response Time | > 2s | SNS Notification |

Set appropriate thresholds based on your application requirements.

## 7. BEST PRACTICES

- Enable detailed monitoring & logging from day one.
- Use centralized logging (CloudWatch / ELK).
- Set meaningful alerts and avoid alert fatigue.
- Monitor critical metrics (CPU, memory, 5xx, latency).
- Use dashboards for quick visibility.
- Keep logs retention as per compliance needs.
- Test alarms and alerting rules regularly.
- Automate monitoring with IaC (Terraform).

## 8. SAMPLE GRAFANA / CLOUDWATCH DASHBOARD



AWS DEVOPS NOTES SERIES

PAGE - 12

# PRODUCTION TROUBLESHOOTING

Detect Fast, Diagnose Right, Resolve Smart

## 1. TROUBLESHOOTING OVERVIEW

- Issues happen – find, diagnose & resolve quickly with minimal impact.
- Helps in application uptime, stability & customer satisfaction.
- Check at Application → Infrastructure → Logs → Metrics.
- Always check recent changes first (deployments, config, infra, code).

Quick detection + Right diagnosis = Faster resolution.

## 83. TROUBLESHOOTING PROCESS

| 1. Detect | 2. Collect Info | 3. Identify Scope | 4. Root Cause | 5. Resolve | 6. Verify |
|---|---|---|---|---|---|
| Alert / User Report | Logs, Metrics, Events | Application / Service / Infra | Find actual issue | Apply fix / Rollback if needed | Confirm fix & Monitor |

Document the issue, RCA & resolution for future reference.

## 3. COMMON AREAS & WHAT TO CHECK

| AREA | WHAT TO CHECK |
|---|---|
| Application / Pods | Pod status (Running/CrashLoop). Application logs, Health check, CPU/Memory |
| Kubernetes (EKS) | Node status, Pod events, Service, Ingress, ConfigMaps, Secrets, Resource usage |
| Deployment | Recent deploys, Image version, Rollback, Environment variables, Config changes |
| Database (RDS) | Connection, CPU/Memory, Slow queries, Active sessions, Storage, Read/Write |
| Cache (ElastiCache) | Connection, Memory usage, Hit/Miss rate, Evictions, Latency |
| Networking (VPC/ALB) | Security Groups, NACL, ALB Target health, DNS, VPC endpoints, Route table |
| AWS Services | CloudWatch metrics, IAM permissions, S3, ECR, ECS, Lambda, etc. |

## 4. USEFUL COMMANDS (EKS / LINUX)

### KUBERNETES COMMANDS (EKS)

```
kubectl get pods -A                 # List all pods
kubectl get nodes                   # Check nodes
kubectl describe pod <pod>          # Pod details
kubectl logs <pod>                  # View logs
kubectl logs <pod> -f               # Follow logs
kubectl get svc -A                  # Services
kubectl get ingress -A              # Ingress resources
kubectl get events -A               # Recent events
kubectl top pods                    # Resource usage (metrics)
```

### LINUX COMMANDS

```
df -h                               # Disk space usage
free -m                             # Memory usage
top / htop                          # Live process view
ps -ef | grep <process>             # Search process
netstat -tulnp                      # Open ports
curl -I <url>                       # Check endpoint
systemctl status <service>          # Check service status
tail -f <log-file>                  # Follow logs
grep -i "error" <log-file>          # Search in logs
```

## 5. COMMON ISSUES & QUICK CHECKS

| ISSUE | CHECK / ACTION |
|---|---|
| Pod Not Running | kubectl describe pod, logs, events |
| High CPU / Memory | kubectl top pods, CloudWatch metrics |
| CrashLoopBackOff | Check logs, env variables, image, probes |
| Database Connection | Check RDS status, security groups, creds |
| Service Not Accessible | Check ALB, Ingress, SG, NACL, DNS |
| High Latency | Check network, DB queries, external APIs |
| Disk Space Issue | df -h, find large files, clean logs |
| Permission Error | Check IAM roles, policies, service account |

## 6. LOG ANALYSIS TIPS

- Search by time range (around issue time).
- Filter by ERROR, WARN, EXCEPTION.
- Correlate logs across services (App, DB, Infra, ALB).
- Compare with previous working deployment.
- Look for patterns (repeated errors, timeouts, high latency).
- Use centralized logging (CloudWatch Logs / ELK).

Key: Don't just check current logs, also look at history and recent changes.

## 7. ESCALATION CHECKLIST

- Identify and document the issue.
- Check if it's a critical outage (user impact).
- Share required details (logs, screenshots, error message, timestamps).
- Notify the right team (Dev / Infra / DB / Cloud).
- Follow escalation matrix & communication channel.
- Keep stakeholders updated until resolution.

## 8. BEST PRACTICES

- Set up proper monitoring & alerting.
- Keep runbooks handy.
- Use dashboards for real-time visibility.
- Monitor key metrics (CPU, Memory, Errors, Latency).
- Automate health checks.
- Perform regular log analysis.
- Follow change management process.
- Have rollback plan ready.

## 9. REMEMBER

- Stay calm and follow a process.
- Check recent changes first.
- Use logs, metrics and events.
- Collaborate with the right team.
- Communicate clearly.
- Document RCA and resolution.
- Optimize to prevent recurrence.

AWS DEVOPS NOTES SERIES

# ROLLBACK & DISASTER SCENARIOS

Be Prepared • Act Fast • Minimize Impact

## 1. ROLLBACK STRATEGIES

| STRATEGY | DESCRIPTION | WHEN TO USE | HOW TO ROLLBACK |
|---|---|---|---|
| K8s Deployment Rollback | Revert to previous ReplicaSet / version | Bad release, failing health checks | • kubectl rollout undo deployment/<name><br>• kubectl rollout history deployment/<name> |
| Jenkins Rollback | Redeploy last successful build | Build / Code issue | • Go to last successful build<br>• Redeploy / Re-run pipeline with that artifact |
| Docker Image Rollback | Use previous stable image from registry | Image causing issues | • Pull previous image tag<br>• Update deployment with stable tag |
| Database Rollback | Restore from backup or point-in-time snapshot | Data corruption, wrong migration | • Restore snapshot (RDS / DB Backup)<br>• Or run down migration script |
| Infrastructure Rollback | Revert infra changes using Terraform | Bad infra change, broken resource | • terraform plan -target=... (as needed)<br>• terraform apply previous known good state |

Always test rollback process before production incidents.

## 2. COMMON ROLLBACK SCENARIOS

- **Application Failure** : New release causes errors → Rollback to last stable version.
- **Performance Issue** : High latency / CPU → Rollback & investigate.
- **Bad Config Change** : Wrong env/config → Revert config / secrets.
- **Failed DB Migration** : Rollback migration or restore backup.
- **Infrastructure Change Failure** : Revert Terraform to previous state.

## 3. DISASTER SCENARIOS

| | |
|---|---|
| Region Outage → | Failover to another region, Route 53 health check & DNS failover. |
| AZ Outage → | Multi-AZ ensures traffic shifts to healthy AZ. |
| Data Loss / Corruption → | Restore from latest backup / Snapshot. Enable automated backups. |
| Application Down → | Auto Scaling + Health Checks + Auto Recovery. |
| Human Error → | Use IaC, GitOps, approvals & change tracking. |

## 4. DISASTER RECOVERY (DR) BEST PRACTICES

Identify Critical Systems & Data → Regular Backups (S3 / EBS / RDS) → DNS Failover (Route 53) → Multi-Region / Multi-AZ Setup → Test DR Plan Regularly → Document & Train Your Team

RTO (Recovery Time Objective) and RPO (Recovery Point Objective) define your DR goals.

## 5. INCIDENT RESPONSE FLOW

1. **Detect** — Alert triggered / issues identified.
2. **Assess** — Understand impact & root cause.
3. **Fix / Rollback** — Apply fix / rollback / workaround.
4. **Restore** — Restore normal service & monitor.
5. **Post-Mortem** — RCA & prevent future incidents.

## 6. CHECKLIST BEFORE ROLLBACK

- [x] Confirm issue is due to latest change.
- [x] Verify last known good version / build.
- [x] Check rollback steps & dependencies.
- [x] Inform team & stakeholders.
- [x] Take backup (DB / Config) if required.
- [x] Execute rollback.
- [x] Validate service health.
- [x] Monitor closely after rollback.
- [x] Update incident ticket / RCA.

## 7. KEY REMINDERS

- Roll forward when possible, rollback when necessary.
- Automate rollback where you can.
- Keep backups & old images / versions.
- Test your DR & rollback regularly.
- Communicate clearly during incidents.
- Document everything.
- Learn from each incident (RCA).

PLAN → PREPARE → PROTECT → PRACTICE

AWS DEVOPS NOTES SERIES

# COMPLETE END-TO-END REAL-WORLD FLOW



Dev Ops

From Developer Commit to Production Monitoring & Feedback Loop

## 1 CODE COMMIT

- Developer writes code
- Push to Git (GitHub)
- Create Pull Request (PR)

## 2 CI PIPELINE (JENKINS)

- Code Checkout
- Build (Maven)
- Run Unit Tests
- Code Quality (SonarQube)
- Build Artifact (JAR/WAR)

## 3 ARTIFACT REPOSITORY

Nexus ECR

- Store build artifacts
- Store Docker images
- Versioned packages
- Artifact traceability

## 4 IMAGE BUILD & PUSH

- Build Docker image
- Tag version
- Push image to ECR / Nexus
- Maintain image versions

## 5 DEPLOY TO QA (EKS)

- Pull image from ECR
- Deploy to QA (EKS)
- Run smoke & integration tests
- Helm / Manifest

## 6 QA VALIDATION & APPROVAL

- Functional Testing
- Performance Testing
- Security Scan
- Bug Fix & Retest
- Approval to Release

## 7 DEPLOY TO PRODUCTION

- Deploy to PROD (EKS)
- Rolling Update
- Zero Downtime
- Monitor & Alert

MONITORING & FEEDBACK LOOP (Continuous Improvement)

## 1. CODE COMMIT

- Developer writes / tests code locally.
- Push changes to Git (GitHub).
- Create Pull Request (PR) for review.
- Use feature branch strategy.

## 2. CI PIPELINE (JENKINS)

- Trigger build on code push.
- Checkout code from Git.
- Build using Maven.
- Run unit tests (JUnit).
- Code quality check (SonarQube).
- Build artifact (JAR/WAR).

## 3. ARTIFACT REPOSITORY

Nexus ECR

- Store build artifacts in Nexus.
- Store Docker images in AWS ECR.
- Artifacts are versioned & traceable.
- Helps in rollback & auditing.

## 4. IMAGE BUILD & PUSH

- Create Dockerfile.
- Build Docker image.
- Tag image with version.
- Push image to ECR / Nexus.
- Use latest/stable tags.

## 5. DEPLOY TO QA (EKS)

- Pull image from ECR.
- Deploy to QA namespace in EKS.
- Use Helm / Kubernetes manifest.
- Run smoke & integration tests.
- Verify application health.

## 6. QA VALIDATION & APPROVAL

- Functional testing (UI + API).
- Performance testing (JMeter).
- Security scan (Trivy, SonarQube).
- Bug fixes & regression testing.
- Once approved, promote to PROD.

## 7. DEPLOY TO PRODUCTION

- Deploy to production namespace in EKS.
- Use rolling update / blue-green.
- Ensure zero downtime.
- Monitor application after release.
- Keep rollback plan ready.

## 8. MONITORING & OBSERVABILITY

**Tools**

- CloudWatch
- ELK Stack
- Prometheus
- AWS X-Ray
- Grafana
- PagerDuty

**Metrics to Monitor**

- CPU & Memory
- Pod / Node Health
- Application Response Time
- Error Rate (5xx)
- Throughput / Traffic
- Business KPIs

**Alerts & Notifications**

- Email (SES)
- Slack
- SMS (SNS)
- PagerDuty

**Auto Actions**

- Auto Scale Pods
- Restart Failed Pods
- Trigger Lambda / Runbook
- Escalate to On-call

## 9. FEEDBACK LOOP




## 10. KEY BENEFITS

- Faster time to market
- High availability & reliability
- Automated & repeatable deployments
- Quick rollback & recovery
- Better visibility & proactive monitoring
- Reduced manual effort
- Continuous improvement

## 11. TOOLS & SERVICES USED

- GitHub
- Jenkins
- Maven
- SonarQube
- Nexus Repository
- Docker
- AWS ECR
- Kubernetes (EKS)
- Helm
- Prometheus
- Grafana
- CloudWatch
- Slack / PagerDuty